LeccionKMeans

November 4, 2026

[4]:
```
import pandas as pd
import matplotlib.pyplot as mp
import seaborn as sb
import os

from sklearn.cluster import KMeans

# Cargar el dataset
dataset = pd.read_csv("Mall_Customers.csv")

# Mostrar las primeras 5 filas
dataset.head()
```

[4]:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

[5]:
```
# Seleccionar las columnas en las posiciones 3 y 4 (índices 3 y 4)
# iloc[:, [3,4]] significa: todas las filas (:), y solo las columnas 3 y 4
fields = dataset.iloc[:, [3, 4]].values

# Imprimir el resultado para verificar los datos
print(fields)
```

```
[[ 15  39]
 [ 15  81]
 [ 16   6]
 [ 16  77]
 [ 17  40]
 [ 17  76]
 [ 18   6]
 [ 18  94]
 [ 19   3]
 [ 19  72]
 [ 19  14]
```

```
[ 19  99]
[ 20  15]
[ 20  77]
[ 20  13]
[ 20  79]
[ 21  35]
[ 21  66]
[ 23  29]
[ 23  98]
[ 24  35]
[ 24  73]
[ 25   5]
[ 25  73]
[ 28  14]
[ 28  82]
[ 28  32]
[ 28  61]
[ 29  31]
[ 29  87]
[ 30   4]
[ 30  73]
[ 33   4]
[ 33  92]
[ 33  14]
[ 33  81]
[ 34  17]
[ 34  73]
[ 37  26]
[ 37  75]
[ 38  35]
[ 38  92]
[ 39  36]
[ 39  61]
[ 39  28]
[ 39  65]
[ 40  55]
[ 40  47]
[ 40  42]
[ 40  42]
[ 42  52]
[ 42  60]
[ 43  54]
[ 43  60]
[ 43  45]
[ 43  41]
[ 44  50]
[ 44  46]
[ 46  51]
```

```
[ 46  46]
[ 46  56]
[ 46  55]
[ 47  52]
[ 47  59]
[ 48  51]
[ 48  59]
[ 48  50]
[ 48  48]
[ 48  59]
[ 48  47]
[ 49  55]
[ 49  42]
[ 50  49]
[ 50  56]
[ 54  47]
[ 54  54]
[ 54  53]
[ 54  48]
[ 54  52]
[ 54  42]
[ 54  51]
[ 54  55]
[ 54  41]
[ 54  44]
[ 54  57]
[ 54  46]
[ 57  58]
[ 57  55]
[ 58  60]
[ 58  46]
[ 59  55]
[ 59  41]
[ 60  49]
[ 60  40]
[ 60  42]
[ 60  52]
[ 60  47]
[ 60  50]
[ 61  42]
[ 61  49]
[ 62  41]
[ 62  48]
[ 62  59]
[ 62  55]
[ 62  56]
[ 62  42]
[ 63  50]
```

```
[ 63  46]
[ 63  43]
[ 63  48]
[ 63  52]
[ 63  54]
[ 64  42]
[ 64  46]
[ 65  48]
[ 65  50]
[ 65  43]
[ 65  59]
[ 67  43]
[ 67  57]
[ 67  56]
[ 67  40]
[ 69  58]
[ 69  91]
[ 70  29]
[ 70  77]
[ 71  35]
[ 71  95]
[ 71  11]
[ 71  75]
[ 71   9]
[ 71  75]
[ 72  34]
[ 72  71]
[ 73   5]
[ 73  88]
[ 73   7]
[ 73  73]
[ 74  10]
[ 74  72]
[ 75   5]
[ 75  93]
[ 76  40]
[ 76  87]
[ 77  12]
[ 77  97]
[ 77  36]
[ 77  74]
[ 78  22]
[ 78  90]
[ 78  17]
[ 78  88]
[ 78  20]
[ 78  76]
[ 78  16]
```

```
[ 78  89]
[ 78   1]
[ 78  78]
[ 78   1]
[ 78  73]
[ 79  35]
[ 79  83]
[ 81   5]
[ 81  93]
[ 85  26]
[ 85  75]
[ 86  20]
[ 86  95]
[ 87  27]
[ 87  63]
[ 87  13]
[ 87  75]
[ 87  10]
[ 87  92]
[ 88  13]
[ 88  86]
[ 88  15]
[ 88  69]
[ 93  14]
[ 93  90]
[ 97  32]
[ 97  86]
[ 98  15]
[ 98  88]
[ 99  39]
[ 99  97]
[101  24]
[101  68]
[103  17]
[103  85]
[103  23]
[103  69]
[113   8]
[113  91]
[120  16]
[120  79]
[126  28]
[126  74]
[137  18]
[137  83]]
```

[6]:
```
import warnings
warnings.filterwarnings("ignore")

# --- Cálculo de la inercia (WCSS) para diferentes números de clústeres ---
wcss = []
for index in range(1, 11):
    # init='k-means++' es el método estándar para inicializar los centroides de␣
 ↪forma inteligente
    kmeans = KMeans(n_clusters=index, init="k-means++", random_state=2)
    kmeans.fit(fields)

    # inertia_ nos da el valor de WCSS (Within-Cluster Sum of Squares)
    wcss.append(kmeans.inertia_)

# --- Graficación del "Codo de Jambu" (Elbow Method) ---
sb.set()  # Configura el estilo de los gráficos de seaborn
mp.plot(range(1, 11), wcss) # En tu imagen usas 'mp' como alias de matplotlib.
 ↪pyplot
mp.title("K-Means Clustering")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS")  # Weighted cluster sum of squares
```

[6]:
```
Text(0, 0.5, 'WCSS')
```



[7]:
```
kmeans = KMeans(n_clusters=5, init="k-means++", random_state=42)

cluster_values = kmeans.fit_predict(fields)

print(cluster_values)
```

```
[2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2
 3 2 3 2 3 2 0 2 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 4 1 4 0 4 1 4 1 4 0 4 1 4 1 4 1 4 1 4 0 4 1 4 1 4
 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1
 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4]
```

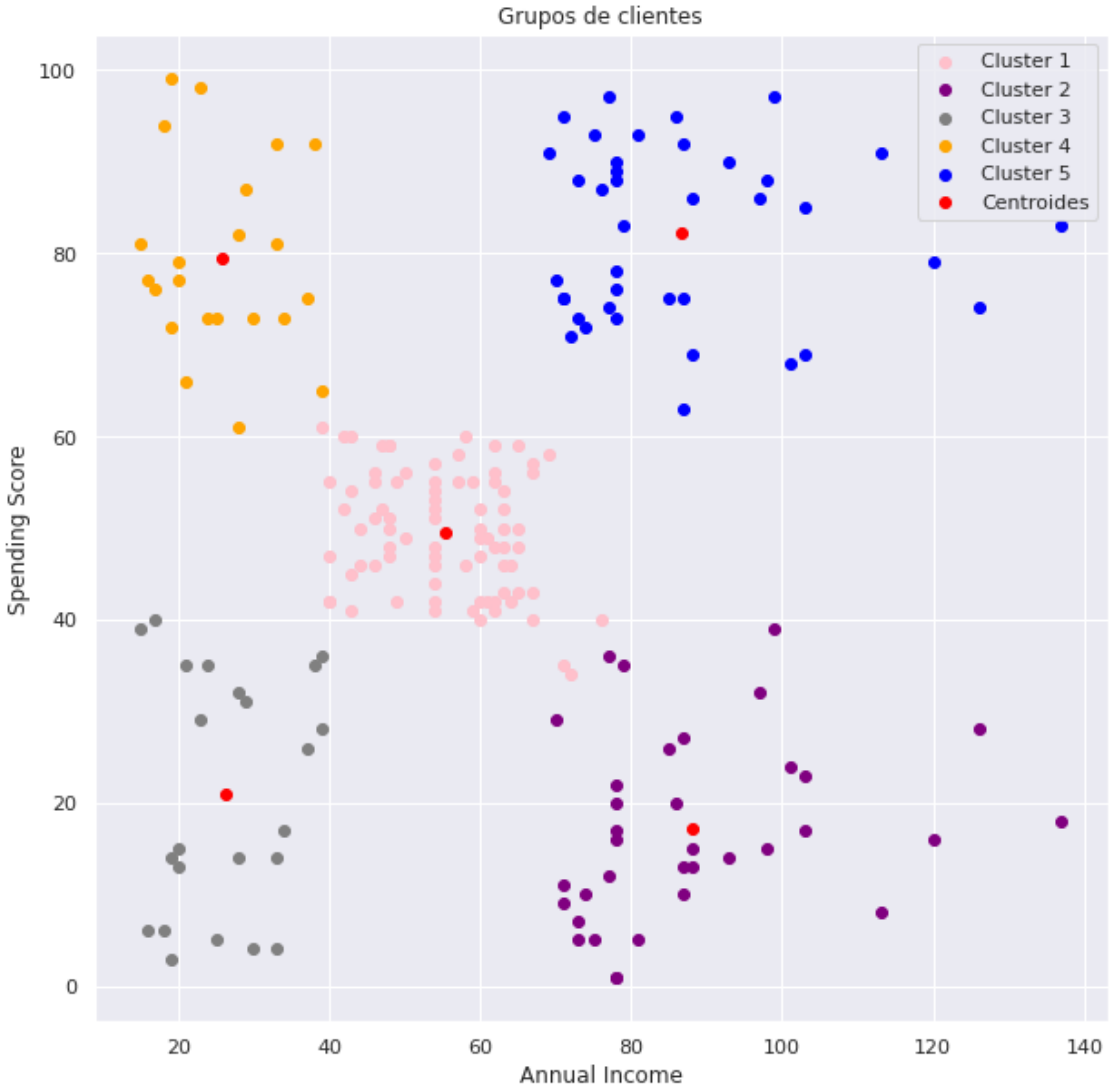
[11]:
```
mp.figure(figsize=(10, 10))

mp.scatter(fields[cluster_values == 0, 0], fields[cluster_values == 0, 1],␣
 ↪c='pink', label='Cluster 1')
mp.scatter(fields[cluster_values == 1, 0], fields[cluster_values == 1, 1],␣
 ↪c='purple', label='Cluster 2')
mp.scatter(fields[cluster_values == 2, 0], fields[cluster_values == 2, 1],␣
 ↪c='gray', label='Cluster 3')
mp.scatter(fields[cluster_values == 3, 0], fields[cluster_values == 3, 1],␣
 ↪c='orange', label='Cluster 4')
mp.scatter(fields[cluster_values == 4, 0], fields[cluster_values == 4, 1],␣
 ↪c='blue', label='Cluster 5')

mp.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],␣
 ↪c='red', label='Centroides')

mp.title("Grupos de clientes")
mp.xlabel("Annual Income")
mp.ylabel("Spending Score")
mp.legend()
```

[11]:
```
<matplotlib.legend.Legend at 0x7181f8c4cf40>
```



[12]:
```python
# Criterio de Silueta ("Silhouette")
from sklearn import datasets
from sklearn.metrics import silhouette_score

X = dataset.iloc[:, [3,4]].to_numpy()

for j in range(2, 12):
    kmeans = KMeans(n_clusters = j, random_state = 42)
    kmeans.fit_predict(X)

    score = silhouette_score(X, kmeans.labels_, metric='euclidean')
    print("Score Silhouette;", "k =", j, ":", score)
```

```
Score Silhouette; k = 2 : 0.2968969162503008
```

```
Score Silhouette; k = 3 : 0.46761358158775435
Score Silhouette; k = 4 : 0.4931963109249047
Score Silhouette; k = 5 : 0.553931997444648
Score Silhouette; k = 6 : 0.53976103063432
Score Silhouette; k = 7 : 0.5264283703685728
Score Silhouette; k = 8 : 0.45827056882053113
Score Silhouette; k = 9 : 0.4565077334305076
Score Silhouette; k = 10 : 0.45925273534781125
Score Silhouette; k = 11 : 0.43844724497454024
```

[13]:
```
#Algoritmo KMeans aplicado a 3 columnas de nuestra base
#bajo una transformación de PCA

fields2 = dataset.iloc[:, [2, 3, 4]].values
print(fields2)
```

```
[[ 19  15  39]
 [ 21  15  81]
 [ 20  16   6]
 [ 23  16  77]
 [ 31  17  40]
 [ 22  17  76]
 [ 35  18   6]
 [ 23  18  94]
 [ 64  19   3]
 [ 30  19  72]
 [ 67  19  14]
 [ 35  19  99]
 [ 58  20  15]
 [ 24  20  77]
 [ 37  20  13]
 [ 22  20  79]
 [ 35  21  35]
 [ 20  21  66]
 [ 52  23  29]
 [ 35  23  98]
 [ 35  24  35]
 [ 25  24  73]
 [ 46  25   5]
 [ 31  25  73]
 [ 54  28  14]
 [ 29  28  82]
 [ 45  28  32]
 [ 35  28  61]
 [ 40  29  31]
 [ 23  29  87]
 [ 60  30   4]
 [ 21  30  73]
```

```
[ 53  33   4]
[ 18  33  92]
[ 49  33  14]
[ 21  33  81]
[ 42  34  17]
[ 30  34  73]
[ 36  37  26]
[ 20  37  75]
[ 65  38  35]
[ 24  38  92]
[ 48  39  36]
[ 31  39  61]
[ 49  39  28]
[ 24  39  65]
[ 50  40  55]
[ 27  40  47]
[ 29  40  42]
[ 31  40  42]
[ 49  42  52]
[ 33  42  60]
[ 31  43  54]
[ 59  43  60]
[ 50  43  45]
[ 47  43  41]
[ 51  44  50]
[ 69  44  46]
[ 27  46  51]
[ 53  46  46]
[ 70  46  56]
[ 19  46  55]
[ 67  47  52]
[ 54  47  59]
[ 63  48  51]
[ 18  48  59]
[ 43  48  50]
[ 68  48  48]
[ 19  48  59]
[ 32  48  47]
[ 70  49  55]
[ 47  49  42]
[ 60  50  49]
[ 60  50  56]
[ 59  54  47]
[ 26  54  54]
[ 45  54  53]
[ 40  54  48]
[ 23  54  52]
[ 49  54  42]
```

```
[ 57  54  51]
[ 38  54  55]
[ 67  54  41]
[ 46  54  44]
[ 21  54  57]
[ 48  54  46]
[ 55  57  58]
[ 22  57  55]
[ 34  58  60]
[ 50  58  46]
[ 68  59  55]
[ 18  59  41]
[ 48  60  49]
[ 40  60  40]
[ 32  60  42]
[ 24  60  52]
[ 47  60  47]
[ 27  60  50]
[ 48  61  42]
[ 20  61  49]
[ 23  62  41]
[ 49  62  48]
[ 67  62  59]
[ 26  62  55]
[ 49  62  56]
[ 21  62  42]
[ 66  63  50]
[ 54  63  46]
[ 68  63  43]
[ 66  63  48]
[ 65  63  52]
[ 19  63  54]
[ 38  64  42]
[ 19  64  46]
[ 18  65  48]
[ 19  65  50]
[ 63  65  43]
[ 49  65  59]
[ 51  67  43]
[ 50  67  57]
[ 27  67  56]
[ 38  67  40]
[ 40  69  58]
[ 39  69  91]
[ 23  70  29]
[ 31  70  77]
[ 43  71  35]
[ 40  71  95]
```

```
[ 59  71  11]
[ 38  71  75]
[ 47  71   9]
[ 39  71  75]
[ 25  72  34]
[ 31  72  71]
[ 20  73   5]
[ 29  73  88]
[ 44  73   7]
[ 32  73  73]
[ 19  74  10]
[ 35  74  72]
[ 57  75   5]
[ 32  75  93]
[ 28  76  40]
[ 32  76  87]
[ 25  77  12]
[ 28  77  97]
[ 48  77  36]
[ 32  77  74]
[ 34  78  22]
[ 34  78  90]
[ 43  78  17]
[ 39  78  88]
[ 44  78  20]
[ 38  78  76]
[ 47  78  16]
[ 27  78  89]
[ 37  78   1]
[ 30  78  78]
[ 34  78   1]
[ 30  78  73]
[ 56  79  35]
[ 29  79  83]
[ 19  81   5]
[ 31  81  93]
[ 50  85  26]
[ 36  85  75]
[ 42  86  20]
[ 33  86  95]
[ 36  87  27]
[ 32  87  63]
[ 40  87  13]
[ 28  87  75]
[ 36  87  10]
[ 36  87  92]
[ 52  88  13]
[ 30  88  86]
```

```
[ 58  88  15]
[ 27  88  69]
[ 59  93  14]
[ 35  93  90]
[ 37  97  32]
[ 32  97  86]
[ 46  98  15]
[ 29  98  88]
[ 41  99  39]
[ 30  99  97]
[ 54 101  24]
[ 28 101  68]
[ 41 103  17]
[ 36 103  85]
[ 34 103  23]
[ 32 103  69]
[ 33 113   8]
[ 38 113  91]
[ 47 120  16]
[ 35 120  79]
[ 45 126  28]
[ 32 126  74]
[ 32 137  18]
[ 30 137  83]]
```

[14]:
```
from sklearn import decomposition

pca = decomposition.PCA(n_components=2)
pca.fit(fields2)
fields2 = pca.transform(fields2)
fields2
```

[14]:
```
array([[-3.18705078e+01, -3.30014253e+01],
       [ 7.63396908e-01, -5.68438653e+01],
       [-5.74087256e+01, -1.31229362e+01],
       [-2.16989648e+00, -5.34779049e+01],
       [-3.21749197e+01, -3.03870050e+01],
       [-2.17827777e+00, -5.22265808e+01],
       [-5.90660565e+01, -9.54170669e+00],
       [ 1.23693261e+01, -6.16176843e+01],
       [-6.63156568e+01, -3.21359934e+00],
       [-5.65677534e+00, -4.72664534e+01],
       [-5.82366179e+01, -9.13375312e+00],
       [ 1.46202592e+01, -6.21072940e+01],
       [-5.51612126e+01, -1.00779778e+01],
       [-4.30660584e-03, -5.01134367e+01],
       [-5.27647474e+01, -1.16804199e+01],
```

```
[ 1.94563480e+00, -5.15231943e+01],
[-3.45062349e+01, -2.37600734e+01],
[-7.30573032e+00, -4.35158724e+01],
[-4.12574933e+01, -1.64734320e+01],
[ 1.61888268e+01, -5.82998775e+01],
[-3.27403118e+01, -2.13349462e+01],
[-9.82702739e-01, -4.44533143e+01],
[-5.78102821e+01, -1.86854579e+00],
[-1.52790692e+00, -4.28591474e+01],
[-5.04821841e+01, -3.56091922e+00],
[ 7.68993306e+00, -4.58611728e+01],
[-3.46334793e+01, -1.50700505e+01],
[-9.94983869e+00, -3.30231964e+01],
[-3.38859638e+01, -1.43425872e+01],
[ 1.33424018e+01, -4.87081562e+01],
[-5.82987123e+01,  4.58075927e+00],
[ 3.30504018e+00, -4.01269208e+01],
[-5.52099698e+01,  6.08913003e+00],
[ 2.05718194e+01, -4.89990472e+01],
[-4.65941080e+01, -1.73866592e-01],
[ 1.13589353e+01, -4.22931022e+01],
[-4.23246581e+01, -2.00398803e+00],
[ 3.95883646e+00, -3.57147310e+01],
[-3.23509213e+01, -5.52987430e+00],
[ 9.18649452e+00, -3.57470830e+01],
[-3.01685636e+01, -6.08872992e+00],
[ 2.23811793e+01, -4.41713774e+01],
[-2.55813646e+01, -8.08067622e+00],
[-2.71889068e+00, -2.46549242e+01],
[-3.20583109e+01, -3.35840237e+00],
[ 1.74791473e+00, -2.78673350e+01],
[-1.04367385e+01, -1.79147281e+01],
[-1.23783039e+01, -1.63356191e+01],
[-1.66862348e+01, -1.32041208e+01],
[-1.70641832e+01, -1.29421904e+01],
[-1.14284717e+01, -1.47072011e+01],
[-2.11691252e+00, -2.13939530e+01],
[-5.86630212e+00, -1.74040262e+01],
[-6.44160070e+00, -1.71804820e+01],
[-1.65307804e+01, -9.75046514e+00],
[-1.91078439e+01, -7.84770642e+00],
[-1.22011311e+01, -1.16806921e+01],
[-1.87466527e+01, -7.02766414e+00],
[-5.70247177e+00, -1.37810191e+01],
[-1.45457835e+01, -7.50635593e+00],
[-9.89837985e+00, -1.10190833e+01],
[-1.04669216e+00, -1.71243951e+01],
```

```
[-1.18868022e+01, -8.30794885e+00],
[-3.92816213e+00, -1.40278916e+01],
[-1.13282609e+01, -7.44952035e+00],
[ 3.46355009e+00, -1.79342631e+01],
[-8.33477343e+00, -9.49491083e+00],
[-1.46311214e+01, -5.07295359e+00],
[ 3.27457589e+00, -1.78032979e+01],
[-8.61404674e+00, -9.21378732e+00],
[-8.91845330e+00, -8.02004253e+00],
[-1.47900012e+01, -3.57136564e+00],
[-1.11560493e+01, -5.07783734e+00],
[-5.65407378e+00, -9.09523241e+00],
[-1.01845040e+01, -8.27472473e-01],
[ 1.55362009e+00, -9.16671924e+00],
[-2.82288621e+00, -6.10446680e+00],
[-5.80799772e+00, -3.88972491e+00],
[ 5.48549687e-01, -8.41178768e+00],
[-1.22247445e+01,  7.32443405e-01],
[-6.66256961e+00, -3.38505720e+00],
[ 7.19261956e-02, -8.16905039e+00],
[-1.64122766e+01,  3.66373064e+00],
[-1.00858289e+01, -8.08279367e-01],
[ 4.85648060e+00, -1.15432860e+01],
[-8.89178433e+00, -1.69417612e+00],
[ 9.83277363e-01, -5.23925550e+00],
[ 4.86143645e+00, -7.83936645e+00],
[ 7.11236959e+00, -8.32897619e+00],
[-6.91516865e+00,  1.80125719e+00],
[-2.65409471e+00, -1.98215661e-01],
[-4.20933575e+00,  1.28831433e+00],
[-3.00194870e+00,  1.43433750e+00],
[-8.56412363e+00,  5.55183811e+00],
[-5.48033702e+00,  3.35628932e+00],
[ 3.89142160e+00, -3.43056812e+00],
[-4.38496751e+00,  2.45119946e+00],
[ 1.75250600e+00, -1.88984535e+00],
[-7.91528320e+00,  6.26010829e+00],
[ 2.87796991e+00, -1.42431246e+00],
[-3.38828369e+00,  4.36826753e+00],
[-2.79963737e+00,  3.75596774e+00],
[ 2.44478856e+00, -1.99708008e-01],
[ 7.04874475e+00, -3.27362700e+00],
[ 3.48833465e+00, -8.35340915e-01],
[-2.22433878e+00,  3.53242354e+00],
[-3.85156474e+00,  5.64292475e+00],
[-4.72786035e+00,  6.36699664e+00],
[-9.73148866e+00,  9.92225023e+00],
```

```
[-5.42355775e+00,  6.79075191e+00],
[-2.09059754e+00,  4.36413238e+00],
[ 8.17420867e+00, -2.80809411e+00],
[-4.25961814e+00,  7.37558345e+00],
[ 2.47487766e+00,  2.59159027e+00],
[ 4.82448589e+00,  2.12117363e+00],
[ 6.20750469e+00,  1.10431167e+00],
[-7.60933562e+00,  1.08841757e+01],
[ 7.61224722e+00, -1.31954477e-01],
[-4.16436318e+00,  1.09293447e+01],
[ 7.02856205e+00,  2.76358934e+00],
[ 1.05889721e+01,  3.25303258e-01],
[-4.06568809e+00,  1.09485378e+01],
[ 1.08815826e+01,  2.49677519e+00],
[ 3.70084414e+01, -1.65733382e+01],
[-8.11111356e+00,  1.77222363e+01],
[ 2.81049250e+01, -8.77789396e+00],
[-6.58597753e+00,  1.77064346e+01],
[ 4.11407352e+01, -1.71212759e+01],
[-2.84734808e+01,  3.35758038e+01],
[ 2.57987536e+01, -5.90493465e+00],
[-2.77777834e+01,  3.31520486e+01],
[ 2.56097794e+01, -5.77396944e+00],
[-3.38179741e+00,  1.67313503e+01],
[ 2.45662280e+01, -3.71766101e+00],
[-2.46421840e+01,  3.35283939e+01],
[ 3.88947579e+01, -1.29277466e+01],
[-2.76055718e+01,  3.55237316e+01],
[ 2.65378878e+01, -3.92614725e+00],
[-1.99345862e+01,  3.13362365e+01],
[ 2.57736097e+01, -2.15096233e+00],
[-3.04569473e+01,  3.99908578e+01],
[ 4.34350999e+01, -1.37876674e+01],
[ 3.12182308e+00,  1.69142673e+01],
[ 3.93077619e+01, -9.53581020e+00],
[-1.77305154e+01,  3.33993277e+01],
[ 4.85122647e+01, -1.49904311e+01],
[-3.21300591e+00,  2.26376014e+01],
[ 2.96784484e+01, -1.26655792e+00],
[-1.09826771e+01,  2.96472544e+01],
[ 4.24650851e+01, -9.37886909e+00],
[-1.66134274e+01,  3.36955092e+01],
[ 3.99482210e+01, -7.57621591e+00],
[-1.44444121e+01,  3.21047336e+01],
[ 3.07052372e+01, -8.20218136e-01],
[-1.81553207e+01,  3.47932836e+01],
[ 4.30019079e+01, -9.72171193e+00],
```

```
       [-2.80555263e+01,  4.20923353e+01],
       [ 3.37890238e+01, -3.01576692e+00],
       [-2.74886037e+01,  4.16994397e+01],
       [ 2.98590413e+01, -1.46199015e-01],
       [-4.33351397e+00,  2.58759881e+01],
       [ 3.84966216e+01, -5.20792430e+00],
       [-1.97440816e+01,  3.98644345e+01],
       [ 4.71559202e+01, -9.06837826e+00],
       [-6.74179118e+00,  3.51056735e+01],
       [ 3.44176762e+01,  5.15039513e+00],
       [-9.35733558e+00,  3.83098091e+01],
       [ 5.12931699e+01, -5.91239638e+00],
       [-2.13287384e+00,  3.43149985e+01],
       [ 2.69188971e+01,  1.31302488e+01],
       [-1.38927217e+01,  4.28736494e+01],
       [ 3.71067519e+01,  5.71942496e+00],
       [-1.54948144e+01,  4.40715294e+01],
       [ 4.89568988e+01, -2.98938430e+00],
       [-1.55717711e+01,  4.52536076e+01],
       [ 4.59634060e+01,  4.76681701e-01],
       [-1.51336232e+01,  4.48915717e+01],
       [ 3.31683881e+01,  9.84031698e+00],
       [-1.31653889e+01,  4.96383291e+01],
       [ 5.11057261e+01,  2.87773203e+00],
       [ 7.49454467e+00,  3.96601531e+01],
       [ 5.08832268e+01,  8.01399365e+00],
       [-6.97952266e+00,  5.14037465e+01],
       [ 5.36107834e+01,  7.28164661e+00],
       [ 1.34179054e+01,  3.77833703e+01],
       [ 6.10844188e+01,  3.05576531e+00],
       [ 3.48575325e-01,  4.97113731e+01],
       [ 3.98457506e+01,  2.10540802e+01],
       [-1.51945356e+00,  5.36429720e+01],
       [ 5.28731796e+01,  1.39620224e+01],
       [ 4.51934486e+00,  4.92827341e+01],
       [ 4.10531324e+01,  2.26207789e+01],
       [-1.19521829e+00,  6.58442298e+01],
       [ 6.30976204e+01,  1.88642286e+01],
       [ 6.56760207e+00,  6.87450641e+01],
       [ 5.83530721e+01,  3.10169261e+01],
       [ 1.99093546e+01,  6.64464251e+01],
       [ 5.85218583e+01,  3.83438527e+01],
       [ 2.09811054e+01,  7.93751463e+01],
       [ 7.24488265e+01,  4.18088331e+01]])
```

[15]:
```
wcss2 = []
for index in range(1, 11):
```

```
    kmeans = KMeans(n_clusters = index, init = "k-means++", random_state = 2)
    kmeans.fit(fields2)

    wcss2.append(kmeans.inertia_)

sb.set()
mp.plot(range(1, 11), wcss2)
mp.title("K-Means Clustering")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS")
mp.show()
```



[16]:
```
kmeans = KMeans(n_clusters=5, init="k-means++", random_state=42)

cluster_values = kmeans.fit_predict(fields2)

print(cluster_values)
```

```
[2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2
 4 2 4 2 4 2 0 2 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 3 1 3 1 0 1 3 1 3 1 3 1 3 1 0 1 3 1 3 1
 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3]
```

```
 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1]
```

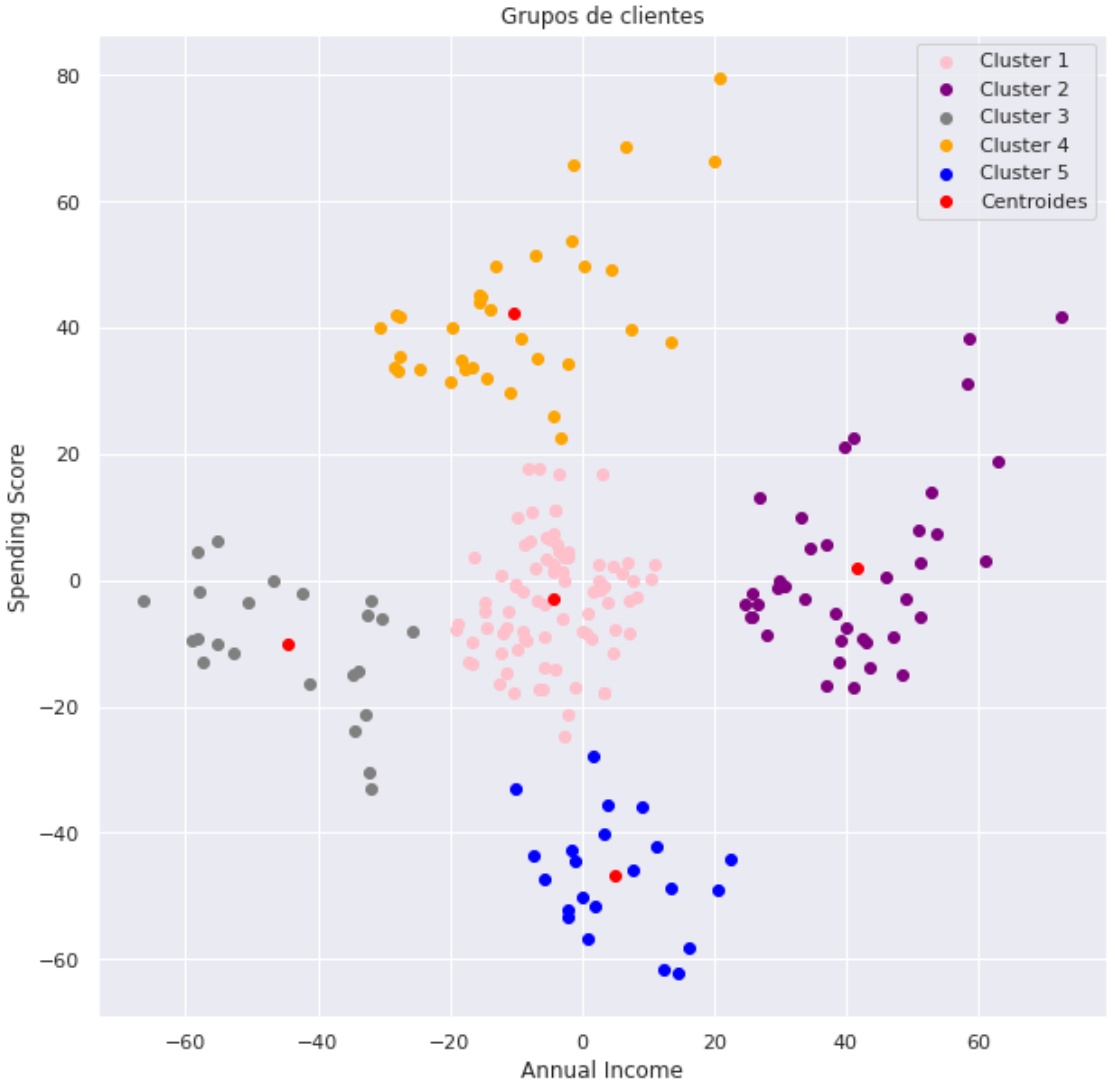
[17]:
```
mp.figure(figsize=(10, 10))

mp.scatter(fields2[cluster_values == 0, 0], fields2[cluster_values == 0, 1],␣
 ↪c='pink', label='Cluster 1')
mp.scatter(fields2[cluster_values == 1, 0], fields2[cluster_values == 1, 1],␣
 ↪c='purple', label='Cluster 2')
mp.scatter(fields2[cluster_values == 2, 0], fields2[cluster_values == 2, 1],␣
 ↪c='gray', label='Cluster 3')
mp.scatter(fields2[cluster_values == 3, 0], fields2[cluster_values == 3, 1],␣
 ↪c='orange', label='Cluster 4')
mp.scatter(fields2[cluster_values == 4, 0], fields2[cluster_values == 4, 1],␣
 ↪c='blue', label='Cluster 5')

mp.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],␣
 ↪c='red', label='Centroides')

mp.title("Grupos de clientes")
mp.xlabel("Annual Income")
mp.ylabel("Spending Score")
mp.legend()
```

[17]:
```
<matplotlib.legend.Legend at 0x7181f8053970>
```


Grupos de clientes
Cluster 1
Cluster 2
Cluster 3
Cluster 4
Cluster 5
Centroides
Spending Score
Annual Income
80
60
40
20
0
−20
−40
−60
−60
−40
−20
0
20
40
60

[ ]: